

Desarrollo Backend

Git, repositorios y Branches
Trabajo colaborativo con GitHub

Objetivos de la clase

- Entender qué problema resuelve Git
- Diferenciar Git de GitHub
- Crear commits y branches
- Usar push, pull y fetch
- Trabajar con Pull Requests
- Resolver conflictos

El problema

4 desarrolladores → una API backend

- ¿Cómo evitamos pisarnos el código?
- ¿Cómo sabemos quién cambió qué?
- ¿Cómo recuperamos una versión anterior?
- ¿Cómo integramos cambios de forma segura?

¿Qué es Git?

Git es un sistema distribuido de control de versiones.

Permite registrar cambios, mantener historial, trabajar en paralelo y recuperar versiones anteriores.

Git ≠ GitHub

GIT

Herramienta de control de versiones.

GITHUB

Plataforma para alojar repositorios Git y colaborar.

Git puede funcionar sin GitHub.

¿Qué es un repositorio?

Un repositorio contiene el proyecto y su historial.

Ejemplo:

Controllers/

Services/

Models/

Tests/

README.md

.gitignore

Crear o clonar un repositorio

Nuevo repositorio:

```
git init
```

Repositorio existente:

```
git clone <url>
```

Ver estado:

```
git status
```

El ciclo de Git

Working Directory

↓ git add

Staging Area

↓ git commit

Local Repository

↓ git push

Remote Repository

git status

Muestra el estado actual del repositorio:

- Archivos modificados
- Archivos nuevos
- Cambios preparados
- Branch actual

Comando:

`git status`

git add

Selecciona cambios para el próximo commit.

```
git add .
```

```
git add Program.cs
```

Importante: add NO crea un commit.

git commit

Registra cambios en el historial local.

```
git commit -m "Add users endpoint"
```

Un buen commit debe explicar qué cambio se realizó.

Historial

Ver commits:

```
git log
```

Versión compacta:

```
git log --oneline
```

Ejemplo:

```
a81f2d3 Add authentication
```

```
91be332 Add users endpoint
```

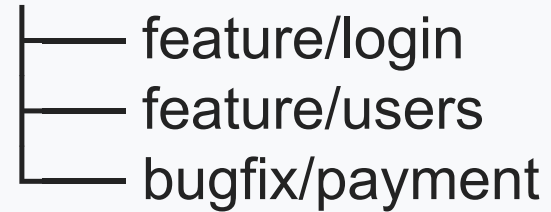
¿Qué es una Branch?

Una branch es una línea independiente de desarrollo.

Permite trabajar en una funcionalidad sin modificar directamente main.

¿Por qué usar Branches?

main



Beneficios:

- Aislar funcionalidades
- Trabajar en paralelo
- Reducir riesgos
- Facilitar code review

Crear y cambiar Branches

Ver branches:

`git branch`

Crear y cambiar:

`git switch -c feature/login`

Cambiar:

`git switch main`

Flujo de una funcionalidad

```
git switch main  
git pull  
git switch -c feature/users
```

desarrollar

```
git add .  
git commit -m "Add users endpoint"  
git push -u origin feature/users
```


Remote, push y pull

Ver remoto:

`git remote -v`

Subir cambios:

`git push`

Traer e integrar cambios:

`git pull`

Fetch vs Pull

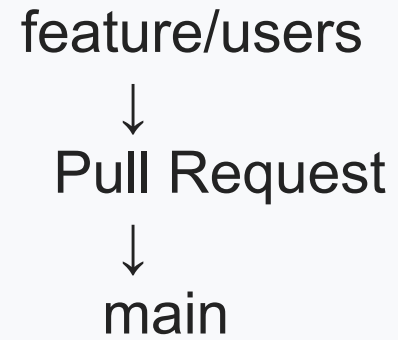
`git fetch`

- Descarga información del remoto.
- No integra automáticamente los cambios.

`git pull`

- Descarga y actualiza la branch actual.

Pull Request



Permite revisar, comentar, ejecutar tests, aprobar y finalmente integrar cambios.

Code Review

Antes de integrar:

- Revisar diseño
- Detectar errores
- Mejorar legibilidad
- Verificar tests

El PR puede recibir comentarios y nuevos commits.

Merge Conflict

Puede ocurrir cuando dos cambios son incompatibles.

```
<<<<<< HEAD  
return Ok(users);  
=====  
return Ok(activeUsers);  
>>>>>> feature/users
```

Resolver un conflicto

1. Leer el conflicto
2. Decidir qué código debe quedar
3. Eliminar los marcadores
4. Guardar
5. `git add .`
6. `git commit`

La decisión depende de la intención del desarrollador.

Flujo profesional

main



switch -c feature/...



code + commits



push



Pull Request



Code Review



merge



main

Práctica guiada

Crear una API .NET

1. Clonar repositorio
2. Crear feature/hello
3. Agregar GET /hello
4. Commit
5. Push
6. Crear Pull Request
7. Revisar el PR de un compañero

Ejercicio evaluable

Product API

Crear:

GET /products

Debe devolver una lista de productos.

Entregables:

- Branch feature/products
- Endpoint funcionando
- Commit claro
- Push
- Pull Request
- README actualizado

Rúbrica

Branch correcta — 1 punto

Endpoint — 2 puntos

Commits claros — 1 punto

Push — 1 punto

Pull Request — 2 puntos

README — 1 punto

Código funcionando — 2 puntos

TOTAL: 10 puntos

Git Cheat Sheet

git clone <url>

git status

git add .

git commit -m "message"

git log --oneline

git branch

git switch -c feature/name

git switch main

git pull

git fetch

git push

git diff

Cierre

Idea clave:

clone → branch → code → add → commit → push → Pull Request → review → merge

A partir de ahora, este flujo será parte de nuestros trabajos backend.